第 6 章練習：CPU 排程

請先畫時間線、列出各工作狀態，再與獨立詳解對照。除非題目另有說明，忽略排程與上下文切換成本。

OS06-E01　非搶占排程計算

單 CPU 上，P1、P2、P3 都在 t＝0 抵達，且各有一次 CPU burst，長度分別為 8、4、2；沒有 I/O。FCFS 同時到達的順序指定為 P1、P2、P3。

1. 畫出 FCFS 與非搶占 SJF 的時間線，依 P1、P2、P3 列出完成、等待及周轉時間。
2. 比較兩者的平均等待時間。
3. SJF 達到此處最短平均等待的結論依賴哪些資訊與模型假設？

OS06-E02　Round Robin

P1、P2、P3 均於 t＝0 抵達，CPU burst 為 5、2、3，初始 ready queue 次序為 P1、P2、P3。沒有 I/O 與切換成本；用完 time quantum 但尚未完成的工作排到隊尾。

1. q＝2 時，畫出時間線。
2. 逐一計算完成時間、等待時間，以及從抵達到首次取得 CPU 的回應時間。
3. 改為 q＝1 後，時間線與上述各量如何改變？區分時間片邊界與實際切換到不同工作的次數。

OS06-E03　優先權與 starvation

假設高優先權工作持續到達，低優先權 batch job L 長期處於 ready。

1. 給出一個讓 L 永遠得不到 CPU 的具體執行序列；單憑一次長時間等待，能否判定 starvation？
2. 設優先權為 0 至 10，數字大者優先；L 從 0 起，每等待 2 ms 提升一級。同級按 FIFO，新到者不能插隊，已被選中的工作每次最多占用 CPU B ms。說明 aging 如何改善此情境。
3. 在 L 升至最高級的時刻，若有 q 個同級 ready 工作排在 L 前面，另有一個 running 工作最多還需 B ms，推導 L 從起始時刻到首次執行的等待上界。若要事先承諾所有輸入的共同上界，還需何種負載限制？

OS06-E04　HRTICK CFS 逐事件推演

使用固定的 Linux 3.2 CFS 教學模型：單 CPU、target latency＝6 ms、minimum granularity＝0.75 ms、wakeup granularity＝1 ms、`GENTLE_FAIR_SLEEPERS=true`，且 `CONFIG_SCHED_HRTICK=y`、runtime feature `HRTICK=true`、`DOUBLE_TICK=false`，CPU 處於 active 狀態，且其 high-resolution timer 已啟用並假設 timer callback 無延遲。P1、P2 於 t＝0 就緒，P3 睡眠至 t＝9；三者 CPU 需求分別為 8、4、2 ms，nice＝0、weight＝1024。三者初始 vruntime 為 0，P1 與 P2 同值時按 P1 先入 tree 的次序選取。假設本題列出的完成、喚醒與排程器事件之外，沒有導致切換的事件；普通 tick 若發生，亦不在這些區間中觸發額外切換。t＝9 同時發生喚醒與原 HRTICK 到期：本題指定先處理 P3 入列及其 `hrtick_update`，再處理原 timer callback；同一時刻事件之間不執行 CPU 工作。

1. P1、P2 都可執行時，計算 period、每人的 ideal runtime 與每執行 1 ms 對 vruntime 的增量。
2. 在 t＝0、3、6、9、11、12、14 各事件處理完並選出下一個 current 後，列出 current、其餘 ready 工作、`min_vruntime` 與切換原因。
3. 畫出完整 Gantt 圖；說明 t＝8 為何不能憑空加入切換。
4. P3 在 t＝9 的 sleep placement 為何？依照題設事件順序，先由哪個排程機制要求重新排程？

OS06-E05　週期 tick 與嚴格大於

沿用 OS06-E04 的工作及初始狀態，但關閉 `CONFIG_SCHED_HRTICK`；一般排程 tick 每 1 ms 發生在整數時刻。

1. 第一個 current 的 `delta_exec` 在 t＝3、4 分別是多少？哪個時刻符合 Linux 3.2 `check_preempt_tick` 的第一條件？
2. 推導新的 Gantt 圖與 P1、P2、P3 的完成時間。
3. 為何 ideal runtime＝3 ms 不能推論一定在 t＝3、6、9 精確切換？
4. 另以 OS06-E04 的 HRTICK 模型為基礎，把 P3 的喚醒時間從 9 改為 8.5 ms；原時間線哪一段先失效？列新的時間線與完成時間。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

OS06-E06　CPU 公平與同步語意

兩個執行緒 A、B 共用一個 CFS runqueue，weight 分別為 1024、335，period＝6 ms。B 先寫入 buffer，再以普通非原子 store 設 `ready=1`；A 輪到時以普通 load 檢查 ready。假設例子的 buffer 由 B 初始化一次，並在 A 完成讀取前保持存活；若要使用發布協定，B 在發布後不再與 A 的讀取並行修改同一資料，否則需另行同步。

1. 計算兩者的 ideal runtime 與 CPU 份額。
2. 較高 weight 能否取代 mutual exclusion？以兩個執行緒各執行一次 `counter++` 說明。
3. 要讓 A 看到 `ready=1` 後可安全讀取已發布的 buffer，需要什麼同步契約與生命週期條件？
4. 用同一把 mutex 保護 `counter++` 提供什麼性質？要發布其他資料，還要確認哪些條件？
5. 說明 memory barrier、atomic operation、mutex 與 semaphore 各自對互斥及資料可見性的作用與限制。

OS06-E07　RM 充分界、EDF 與短期限

單 CPU 可立即搶占，沒有 blocking、自我暫停、I/O 或額外成本。每個 task 自 t＝0 同步開始，之後每 T 個時間單位週期性釋放一個 job；c 是每個 job 實際且最壞的 CPU 需求。完成恰在期限算成功，逾期 job 不丟棄；同 task 的舊 job 先執行。EDF 平手時保留 current，否則依 task／job 編號。

甲組 τ_1＝(C＝2,T＝D＝5)、τ_2＝(C＝4,T＝D＝8)。

1. 算利用率 U 與兩 task 的 RM 充分界；能否只靠此測試判定成功或失敗？以 $w^{(0)}=C_2$、$w^{(k+1)}=C_2+\lceil w^{(k)}/T_1\rceil C_1$ 迭代。
2. 列出 RM 在 [0,10) 的每段執行、完成與期限，核對前題固定點。
3. 改用 EDF，列出 [0,16) 每次釋放／完成後的選擇、剩餘需求與絕對期限；核對所有在 16 或之前到期的 job。有限前綴通過後，哪個定理可保證全部後續工作？
4. 乙組 τ_1＝(1,2,2)、τ_2＝(2,4,4)，畫出一個共同週期的 RM 時間線，說明超過充分界不必然失敗。
5. 丙組兩個 task 均為 C＝2、T＝10、D＝3。以長度 3 的區間需求說明為何任何政策都失敗；比較甲、丙兩組能否僅以 U 判定 EDF。若甲組另有鎖阻塞，原結論是否仍足夠？

OS06-E08　到達不一定引起搶占

單 CPU、無 I/O、切換成本為 0；工作（到達時間，CPU 需求）為 P1＝(0,7)、P2＝(2,3)、P3＝(4,1)，單位為 ms。SRTF 只在新工作剩餘時間嚴格較短時搶占；剩餘相等時保留 current，其餘平手依工作編號。

1. 畫出 SRTF 與非搶占 SJF 的逐段時間線。
2. 依 P1、P2、P3 的順序，計算完成、周轉、等待和首次 CPU 回應時間。
3. t＝4 是否切換？比較兩政策的平均等待時間，說明為何到達事件本身不等於搶占。

OS06-E09　MLFQ 被打斷的時間額度

單 CPU；新工作進 Q0，Q0 使用 RR、q＝2，Q1 使用 RR、q＝4，Q2 使用 FCFS。較高層可搶占，同層新到者不搶占；用盡額度但尚未完成才降級。被高層打斷者回到原層隊首並保留未用額度。無 I/O、無提升、成本為 0。A＝(到達 0，需求 7)、B＝(1,2)、C＝(5,1)，單位 ms。

1. 列出每段執行的層級、剩餘需求與剩餘額度，特別標示 t＝5、6、9。
2. 計算各工作完成、等待與首次 CPU 回應時間。
3. 若在 t＝6 把 A 的 Q1 額度重設為 4，時間線如何改變？為何這是不同政策？

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study